Information Technologies for Industrial Engineers

Vue Basics

Why Vue?

- Easy to start: write mostly HTML, CSS, and JavaScript.
- Clear syntax for beginners.
- Great for building interactive web pages.

Vue App Structure

`main.js`

```
import { createApp } from "vue";
import App from "./App.vue";

createApp(App).mount("#app");
```

- `createApp(...)` creates your app.
- `.mount("#app")` tells Vue where to render on the page.

Template Syntax

App.vue

```
<script setup lang="ts">
const msg = "Hello World";
const num = 1;
const birthYear = 2000;
const currentYear = new Date().getFullYear();
</script>

<template>
  <div>This is just like HTML.</div>
  <div>{{ 1 + 1 }}</div>
  <div>{{ msg }}</div>
  <div>{{ num + 1 }}</div>
  <div>I am {{ currentYear - birthYear }} years old.</div>
</template>
```

Style

App.vue

```
<template>
  <div class="wrapper">
    <h2 class="header">Test CSS</h2>
  </div>
</template>

<style scoped>
.wrapper {
  margin: 0.5em auto;
  width: 95vh;
}
.header {
  color: teal;
}
</style>
```

Event Handling

`App.vue`

```
<script setup>
function handleClick() {
  alert("Hello!");
}
</script>

<template>
  <button @click="handleClick">Click</button>
</template>
```

- `@click` listens for click events.
- Other events: `@mouseover`, `@keydown`, `@submit`, etc.

Component

- Components are reusable UI blocks.
- Each component has its own template, script, and style.

components/NavBar.vue

```vue
<template>
  <div class="nav">
    <span>Home</span>
    <span>About Me</span>
  </div>
</template>

<style scoped>
.nav {
  display: flex;
  gap: 0.5em;
  color: teal;
  background-color: rgb(243, 243, 243);
  padding: 0.2em 0.5em;
}
</style>
```

App.vue

```
<script setup>
import NavBar from "./components/NavBar.vue";
</script>

<template>
  <NavBar />
  <div class="wrapper">...</div>
</template>
```

State

Counter App (V1)

```
<script setup>
let counter = 0;
function handleClick() {
  counter += 1;
  console.log(counter);
}
</script>

<template>
  <div>{{ counter }}</div>
  <button @click="handleClick">Add</button>
</template>
```

Counter App (V1)

- Why doesn't it work? If you console.log `counter`, it will show the correct value, but the template doesn't update.

`ref(...)` for Reactive State

- Use `ref(...)` to create reactive state.
- `ref(...)` returns an object with a `.value` property.
- In template, you can use the `ref` directly without `.value`.
- In JavaScript, you must use `.value` to access the value.

Counter App (V2)

```
<script setup>
import { ref } from "vue";

const counter = ref(0);

function handleClick() {
  counter.value += 1;
}
</script>

<template>
  <div>{{ counter }}</div>
  <button @click="handleClick">Add</button>
</template>
```

Common Mistake

- In JavaScript, use `counter.value`.
- In template, use `{{ counter }}`.

Form App (`v-model`)

```
<script setup>
import { ref } from "vue";

const name = ref("");
</script>

<template>
  <input v-model="name" placeholder="Type your name" />
  <p>Hello, {{ name }}</p>
</template>
```

- `v-model` keeps input and state in sync.

Todo App Idea

- Create `todos` list using `ref([])`.
- Add new item from an input box.
- Show list with `v-for`.
- Prevent adding empty text.

```
<li v-for="(todo, index) in todos" :key="index">{{ todo }}</li>
```

Summary

- Template syntax: `{{ ... }}`
- Events: `@click`
- Reactive state: `ref(...)`
- Two-way binding: `v-model`
- List rendering: `v-for`